

2. Variables, Data Type, Scope

Author -- Yucheng Wu

Version -- 26.1.25

Example 2.1

```
/**
 * TicketMachine models a ticket machine that issues
 * flat-fare tickets.
 * The price of a ticket is specified via the constructor.
 * Instances will check to ensure that a user only enters
 * sensible amounts of money, and will only print a ticket
 * if enough money has been input.
 *
 * @author David J. Barnes and Michael Kölling
 * @version 7.1
 */
public class TicketMachine
{
    // The price of a ticket from this machine.
    private int price;
    // The amount of money entered by a customer so far.
    private int balance;
    // The total amount of money collected by this machine.
    private int total;

    /**
     * Create a machine that issues tickets of the given price.
     */
    public TicketMachine(int cost)
    {
        price = cost;
        balance = 0;
        total = 0;
    }

    /**
     * @Return The price of a ticket.
     */
    public int getPrice()
    {
        return price;
    }

    /**
     * Return The amount of money already inserted for the next ticket.
     */
    public int getBalance()
    {
        return balance;
    }

    /**
     * Receive an amount of money from a customer.
     */
}
```

```

    * Check that the amount is sensible.
    */
    public void insertMoney(int amount)
    {
        if(amount > 0) {
            balance = balance + amount;
        }
        else {
            System.out.println("Use a positive amount rather than: " + amount);
        }
    }

    /**
     * Print a ticket if enough money has been inserted, and
     * reduce the current balance by the ticket price. Print
     * an error message if more money is required.
     */
    public void printTicket()
    {
        if(balance >= price) {
            // Simulate the printing of a ticket.
            System.out.println("#####");
            System.out.println("# The BlueJ Line");
            System.out.println("# Ticket");
            System.out.println("# " + price + " cents.");
            System.out.println("#####");
            System.out.println();

            // Update the total collected with the price.
            total = total + price;
            // Reduce the balance by the price.
            balance = balance - price;
        }
        else {
            System.out.printf("You must insert at least %d more cents.%n",
                             price - balance);
        }
    }

    /**
     * Return the money in the balance.
     * The balance is cleared.
     */
    public int refundBalance()
    {
        int amountToRefund;
        amountToRefund = balance;
        balance = 0;
        return amountToRefund;
    }
}

```

2.1. Variables

Definition

Variables are **named storage locations** for data that a program uses.

Features

- Must be **declared** with a type before use.
- Can be initialized with a value.

Example

```
int age = 20;  
String name = "Alice";
```

Explanation

- `int age` declares an integer variable.
 - `String name` declares a string variable.
-

2.1.1. Instance Variables

Definition

Variables declared **inside a class but outside any method or constructor**. Also called **fields**.

Features

- Each object (instance) has its **own separate copy**.
- Initialized to default values if not explicitly set.

Example

```
class Student {  
    String name; // instance variable  
    int age;     // instance variable  
  
    void printInfo() {  
        System.out.println(name + " is " + age + " years old.");  
    }  
}
```

Explanation

- `name` and `age` are instance variables.
- Each `Student` object will have its own `name` and `age`.

Remark

- Their lifetime is tied to the **object's lifetime**.
-

2.1.2. Class (Static) Variables

Definition

Variables declared with the `static` **keyword** inside a class but outside any method or constructor.

Features

- Shared among all instances of the class.
- Only **one copy** exists regardless of the number of objects created.

Example

```
class Student {  
    static int count = 0; // static (class) variable  
  
    Student() {  
        count++;  
    }  
}
```

Explanation

- `count` keeps track of the total number of `Student` objects created.
- All `Student` objects **share the same** `count` variable.
- I will introduce the meaning of `static` key word in future.

Remark

- Accessed using `ClassName.variableName` syntax (e.g. `Student.count`).
-

2.1.3. Local Variables

Definition

Variables declared **inside methods, constructors, or blocks**.

Features

- Only accessible **within the method or block** where declared.
- **Must be initialized** before use.

Example

```
void printSum() {  
    int a = 5; // local variable  
    int b = 10; // local variable  
    System.out.println(a + b);  
}
```

Explanation

- `a` and `b` exist only while `printSum()` is executing.
- They are destroyed once the method finishes.

Remark

- Local variables do **not have default values**; uninitialized use causes compilation error.
-

2.1.4. Parameters

Definition

Variables declared in a **method's signature** to receive values passed when the method is called. Also known as **formal parameters**.

Features

- Act as **local variables** within the method body.
- Must be declared with a type and name in the method parentheses.
- Values are passed from **arguments** (actual values) during method invocation.

Example

```
/**
 * Receive an amount of money from a customer.
 * Check that the amount is sensible.
 */
public void insertMoney(int amount)
{
    if(amount > 0) {
        balance = balance + amount;
    }
    else {
        System.out.println("Use a positive amount rather than: " + amount);
    }
}
```

Explanation

- `num1` and `num2` are parameters that receive values when `add` is called.
- They are only accessible inside the `add` method.
- Parameters allow methods to operate on different data without hardcoding values.

Remark

- Parameters do **not have default values** unless specified (e.g., in Java 8+ for default methods in interfaces).
- Changes to parameters inside the method do not affect the original arguments (pass-by-value).

Summary Remark

- **Instance variables:** per object (fields)
 - **Static (class) variables:** per class (shared)
 - **Local variables:** per method/block (temporary)
-

2.2. Conditional Composition (if, if-else)

Definition

Used to perform *different actions based on boolean conditions*.

Features

- Uses boolean expressions that evaluate to `true` or `false`.
- Can be **nested** to check multiple conditions.

Example

```
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Explanation

- If `age >= 18` is true, prints "Adult"; otherwise prints "Minor".

Notation: "||", "&&"

- "||" refer to "or"; "&&" refer to "and".
-

2.3. Scope and Lifetime

Definition

Scope determines where a variable can be **accessed**, while lifetime determines how long it **exists** in memory.

Types

- **Class-level (fields):** exist as long as the object exists (defined by fields).
- **Method-level (local variables):** exist during method execution.
- **Block-level:** exist only within the block `{ }` they are declared.

Example

```
public void exampleMethod() {  
    int x = 5; // x exists only within this method  
    if (x > 0) {
```

```
        int y = 10; // y exists only within this if block
    }
}
```

Explanation

- `x` is method-level, `y` is block-level.